Zuu Crew
Machine Learning
Acadamy

Apache Kafka Integration

Building Production Ready ML Systems

Zuu Crew Machine Learning Academy

September 2025

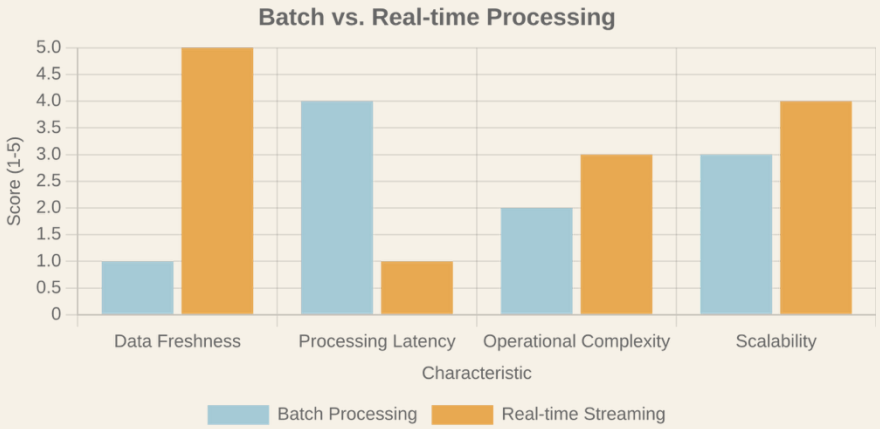
The ML Data Challenge

Problem Statement

Bridging the Gap

Modern machine learning pipelines face significant challenges in handling the volume, velocity, and variety of data required for effective model training and inference.

- **Data Freshness** - Models need access to the most recent data for accurate predictions
- **Scalability** - Systems must handle growing data volumes without redesign
- **Consistency** - Training and serving environments need the same data processing
- **Reliability** - Production ML requires fault-tolerant data pipelines



Traditional Approaches Fall Short

Batch Processing: Introduces latency, limiting real-time applications

Point-to-Point Integration: Creates brittle, hard-to-maintain systems

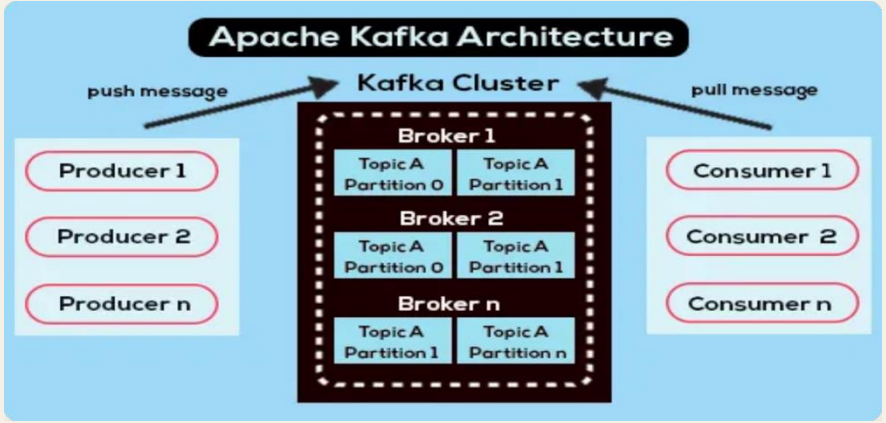
Manual ETL: Doesn't scale with growing data needs

Separate Training/Serving: Leads to training/serving skew

Why Kafka?

Apache Kafka serves as the ideal event backbone for real-time machine learning systems, enabling seamless data flow from source to model to action.

- **High Throughput:** Handles millions of events per second
- **Durability:** Persistent storage with configurable retention
- **Replayability:** Ability to reprocess historical data



- **Scalability:** Horizontal scaling
- **Fault Tolerance:** Built-in replication
- **Decoupling:** Flexible architecture
- **Integration:** Bronze–Silver–Gold ready

Kafka in Our Project

Our project demonstrates Kafka's capabilities in a real-time churn prediction pipeline, showcasing how it enables continuous data flow and model serving.

1 Data Generation

Producer generates customer events from the ChurnModelling dataset and publishes to the **customer_events** topic.

2 Data Storage

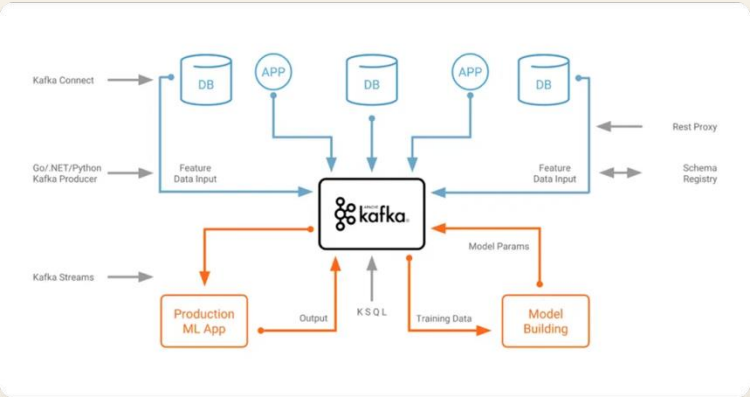
Kafka stores events with configurable retention, enabling replay capabilities for model retraining and testing.

3 Processing

Spark Structured Streaming consumes events, applies transformations, and writes to Bronze storage layer.

4 Prediction & Orchestration

ML model produces predictions to **churn_predictions** topic, while Airflow orchestrates the entire workflow.

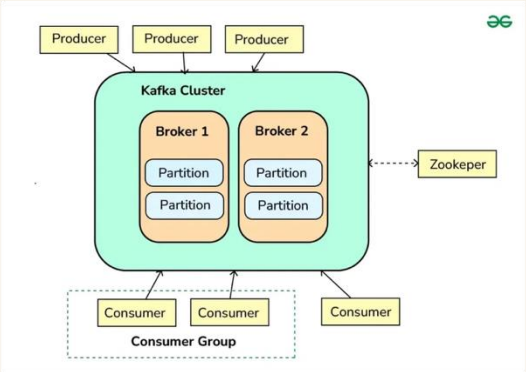


Kafka Core Components

Architecture Overview

Building Blocks of Kafka

Apache Kafka is a distributed streaming platform built around five core components that work together to enable high-throughput, fault-tolerant, real-time data streaming pipelines.



Kafka in ML Pipelines

Data Ingestion: Producers send events to Kafka topics

Stream Processing: Spark/Flink consume and process streams

Core Components

Producer: Client application that publishes events to Kafka topics

Broker: Server that stores data and serves client requests

Topic: Named feed or category of events where data is stored

Consumer: Client application that subscribes to topics and processes events

Consumer Group: Set of consumers that cooperatively consume from topics

Component	Role in ML Pipeline
Producer	Generates events (e.g., user interactions, sensor data)
Topic	Stores events by category (e.g., customer_events)
Consumer	ML model or data processor that reads from topics

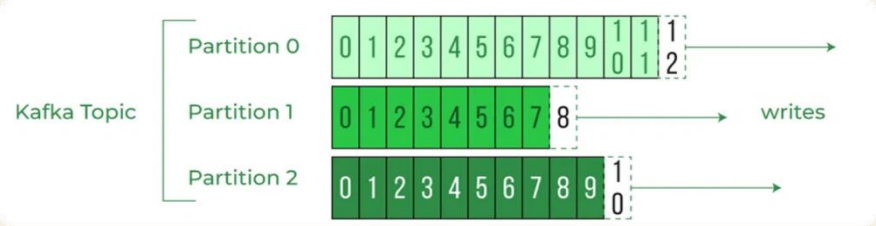
Topics & Partitions

Core Concepts

Stream Organization

Topics and partitions form the fundamental organizational structure in Kafka, enabling scalable and ordered event streaming.

- **Topic = Stream of Events** - A named feed or category of messages
- **Partition = Ordered Log** - Each topic is split into ordered, immutable partitions
- **Offsets Ensure Ordering** - Each record gets a sequential ID (offset)
- **Enables Scalability** - Partitions allow distribution across multiple brokers



Topic Examples

ML Use Case: customer_events topic

Partition Key: Using customer ID

Scaling

Throughput: More partitions = more parallel consumers

Retention: Configurable per topic

Consumer Groups

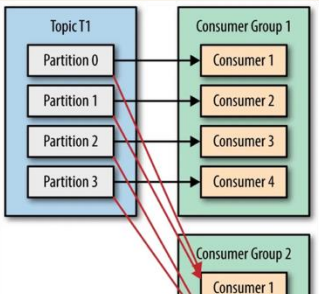
Scalable Consumption

Distributing Load Across Consumers

Consumer groups enable horizontal scaling by distributing partition processing across multiple consumer instances, providing fault tolerance and load balancing.

Key Principles of Consumer Groups

Each partition is consumed by exactly one consumer in a group
Consumers can belong to different groups for different use cases
Automatic rebalancing when consumers join or leave
Each group maintains its own offset position per partition



Scenario	Consumer Group Behavior
More consumers than partitions	Some consumers will be idle (partitions cannot be shared)
Equal consumers and partitions	Optimal 1:1 mapping for maximum parallelism
Fewer consumers than partitions	Some consumers handle multiple partitions
Consumer failure	Automatic rebalancing assigns partitions to remaining consumers

ML Pipeline Applications

Consumer groups enable critical ML capabilities:

Parallel Processing: Scale out feature extraction across multiple instances

Multiple Use Cases: Different consumer groups for training, inference, and monitoring

Fault Tolerance: Automatic recovery if model serving instances fail

ZooKeeper in Kafka

Coordination Service

ZooKeeper's Role

Apache ZooKeeper is a centralized service for maintaining configuration information, naming, providing distributed synchronization, and group services for Kafka clusters.

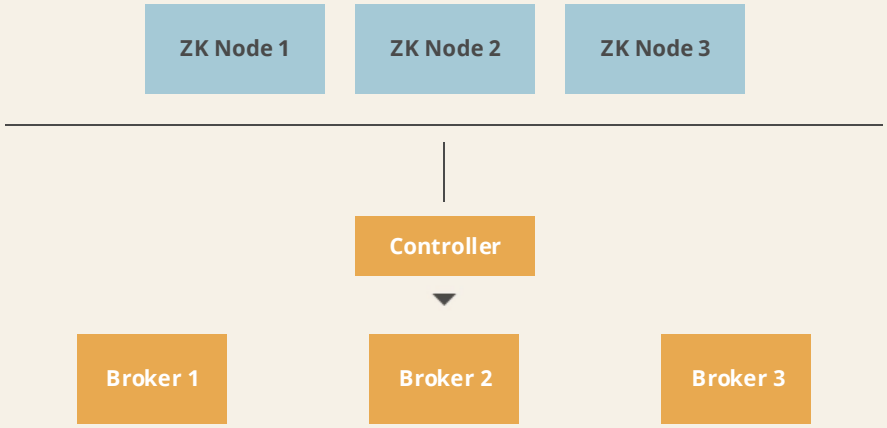
- **Broker Management** - Tracks broker status and maintains the list of brokers in the cluster
- **Topic Configuration** - Stores metadata about topics, partitions, and their configurations
- **Leader Election** - Manages leader election for partition replicas when brokers fail
- **Access Control** - Stores ACLs for Kafka resources when security is enabled

ZooKeeper Paths

/brokers/ids - Broker registration information

/brokers/topics - Topic metadata and partition assignments

/controller - Controller broker election and epoch



Aspect	ZooKeeper's Role
Ensemble	Typically 3, 5, or 7 nodes for high availability
Consensus	ZAB (ZooKeeper Atomic Broadcast) protocol
Scalability	Limited to ~100K partitions in large clusters
Complexity	Requires separate system to manage and maintain

Limitations

ZooKeeper dependency introduces operational complexity and scalability limitations, leading to the development of KRaft mode as a ZooKeeper-free alternative.

KRaft Mode

ZooKeeper-Free Kafka

Kafka Raft Metadata Mode

KRaft (Kafka Raft) mode is a new architecture introduced in Kafka 2.8 that eliminates the ZooKeeper dependency by implementing the Raft consensus protocol directly within Kafka.

- **Simplified Architecture** - Single system for both data storage and metadata management
- **Improved Scalability** - Supports larger clusters with more partitions and topics
- **Enhanced Performance** - Faster controller failover and topic creation operations
- **Reduced Operational Complexity** - Single system to monitor, configure, and maintain

Configuration

```
process.roles=broker,controller
controller.quorum.voters=1@localhost:9093
listeners=PLAINTEXT://:9092,CONTROLLER://:9093
```

- Combined mode
- Controller quorum
- Dual ports

Combined Node 1
Broker + Controller
9092 (Client) / 9093 (Controller)

Combined Node 2
Broker + Controller
9092 (Client) / 9093 (Controller)

Combined Node 3
Broker + Controller
9092 (Client) / 9093 (Controller)

Feature	ZooKeeper Mode	KRaft Mode
Components	ZK + Kafka	Kafka only
Metadata Storage	ZooKeeper	Kafka internal
Controller Failover	Slower	Faster
Scalability Limit	~100K partitions	~1M+ partitions

Production Status

KRaft mode became production-ready in Kafka 3.3.1 (October 2022) and is now the recommended deployment mode for new Kafka clusters, especially for ML pipelines requiring high scalability.

Kafka vs Traditional Queues

Kafka's architecture differs fundamentally from traditional message queues, offering unique advantages for ML pipelines.

Traditional Message Queues

Point-to-point messaging systems designed for task distribution.

Message Lifecycle

Messages are deleted after being consumed, preventing replay

Consumer Model

One-to-one: Each message goes to exactly one consumer

Scalability

Limited by queue-based architecture, often requires sharding

Use Cases

Task distribution, workload balancing, request-response patterns

ML Pipeline Limitations:

"Once a prediction request is processed, the data is gone. Historical retraining requires separate storage solutions and introduces complexity."

Apache Kafka

Distributed commit log designed for high-throughput event streaming.

Message Lifecycle

Messages are retained for configurable period, enabling replay

Consumer Model

One-to-many: Multiple consumer groups read independently

Scalability

Highly scalable through partitioning and distributed architecture

Use Cases

Event streaming, log aggregation, metrics, real-time analytics

ML Pipeline Advantages:

"Data scientists can reprocess historical events for model retraining while production systems simultaneously consume the same data for real-time inference."

Zuu Crew

Delivery Guarantees

Kafka provides three levels of message delivery guarantees, each with different trade-offs between performance and reliability. Understanding these guarantees is crucial for designing ML pipelines with appropriate data consistency requirements.

Guarantee Type	Message Delivery	ML Pipeline Use Case
At most once	Messages may be lost but never duplicated	High-volume metrics where some loss is acceptable
At least once	Messages are never lost but may be duplicated	Feature extraction where duplicates can be filtered
Exactly once	Messages are processed exactly one time	Critical ML inference where accuracy is paramount

At Most Once

Producer sends message and doesn't wait for acknowledgment. Fastest but least reliable option.

At Least Once

Producer retries until acknowledgment is received. Most common choice balancing reliability and performance.

Exactly Once

Achieved with transactional APIs and idempotence. Critical for financial applications and precise ML model training.

Why Kafka for ML?

Apache Kafka provides unique capabilities that make it an ideal foundation for modern machine learning pipelines, enabling real-time data flow and model serving.

Real-time Ingestion

Enables continuous data streaming for fresh feature extraction and immediate model updates, critical for time-sensitive applications like fraud detection.

Replay for Retraining

Historical data can be replayed from Kafka topics to retrain models with consistent datasets, ensuring reproducible ML experiments and model versioning.

Integration Ecosystem

Seamless integration with Spark, Flink, and Airflow creates an end-to-end ML pipeline from data ingestion to model deployment and monitoring.

Streaming ML

Enables online learning and real-time prediction serving, allowing models to continuously adapt to changing data patterns without batch retraining cycles.

Zuu Crew

Kafka + Airflow Orchestration

Apache Airflow provides powerful workflow orchestration capabilities that complement Kafka's streaming architecture, enabling end-to-end ML pipeline management.

Key Integration Points

Airflow DAGs manage Kafka topic lifecycle
Operators trigger Spark streaming jobs
Sensors monitor for new data in topics
Scheduled tasks handle data compaction

Orchestration Workflow Categories

Topic Management

Create, configure, and monitor Kafka topics

Data Compaction

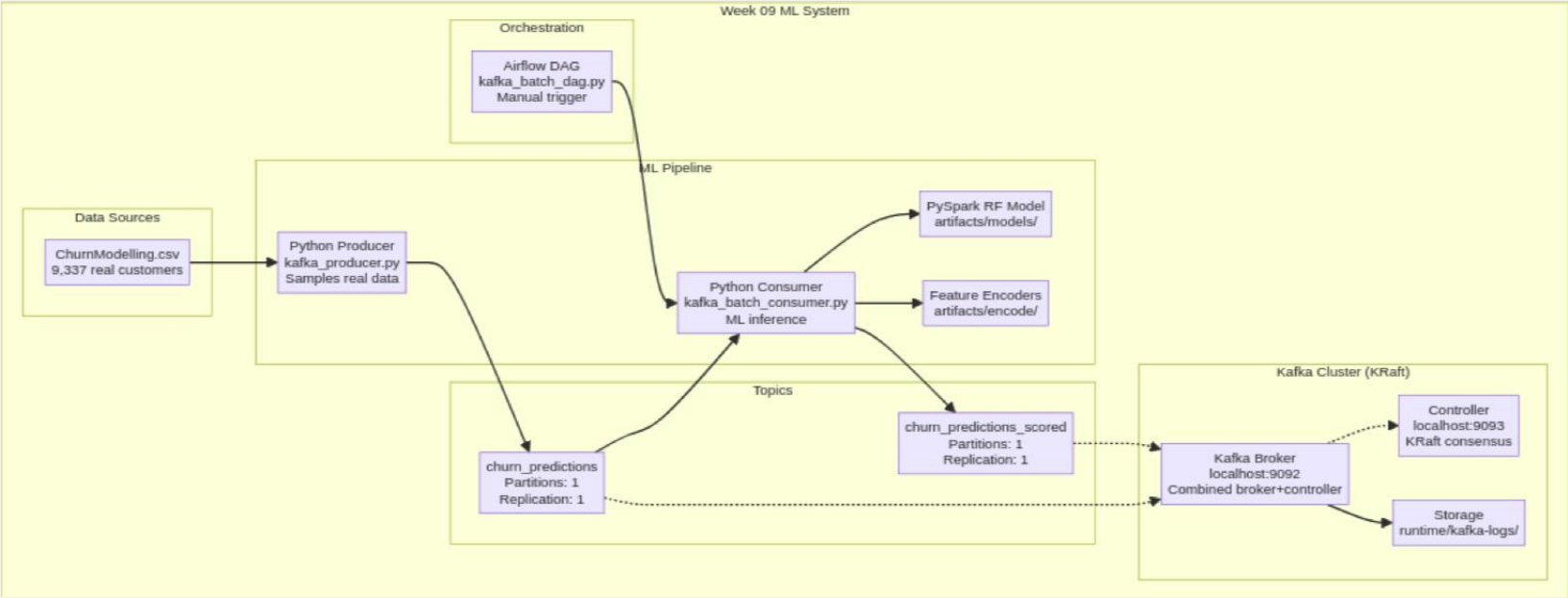
Schedule daily jobs to compact streaming data

Model Training

Trigger periodic model retraining

Monitoring

Track consumer lag and throughput metrics

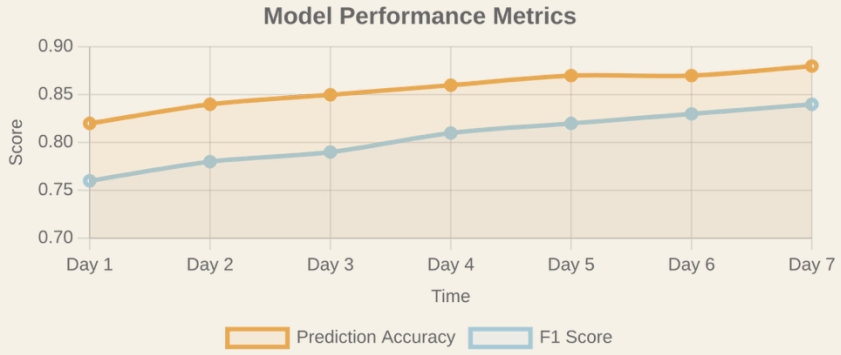


Churn Prediction Use Case

Practical Implementation

Real-time Customer Churn Prediction

Our project demonstrates how Kafka enables real-time churn prediction by continuously processing customer behavior events and delivering immediate insights for retention actions.



Implementation Components

Event Producer: Simulates customer interactions and publishes to Kafka topic

Kafka Topics: customer_events for raw data, churn_predictions for model outputs

Spark Streaming: Processes events in micro-batches, extracts features in real-time

ML Model: RandomForest classifier trained on historical data, deployed for streaming inference

Monitoring: Tracks prediction accuracy, feature drift, and model performance

Pipeline Stage	Kafka's Role
Data Ingestion	Captures customer events with exactly-once delivery
Feature	Enables real-time feature extraction from

Data Flow Walkthrough

Our Kafka-based ML pipeline follows a clear data flow path from raw events to actionable predictions, demonstrating how streaming data can power real-time machine learning systems.

Key Data Flow Principles

End-to-end streaming architecture with minimal batch processing

Decoupled components communicate via Kafka topics

Data quality checks at each transformation stage

Exactly-once processing guarantees for critical operations

Comprehensive logging and monitoring throughout the pipeline

Performance Metrics

Our pipeline achieves end-to-end latency of under 2 seconds from event generation to prediction availability, with throughput capacity of 10,000 events per second and 99.9% uptime.

Step-by-Step Data Flow

1. Event Generation

Customer events from the ChurnModelling dataset are generated and published to the customer_events topic

2. Stream Ingestion

Spark Structured Streaming consumes events from Kafka with schema validation and initial transformations

3. Storage

Raw events are persisted to storage layer with metadata enrichment and partitioning

4. Transformation

Airflow triggers data cleaning, normalization, and feature engineering jobs to create feature tables

5. Model Inference

RandomForest model reads from feature tables and generates churn probability scores for each customer

6. Prediction Publishing

Predictions are published to the churn_predictions Kafka topic for downstream consumption